

Tujitunze / HSIMS

Health Savings and Insurance Management System for Tanzania — a member-owned digital wallet that turns everyday mobile-money, airtime and bank activity into a traceable contribution toward each member's own health cover.

Document	System Requirements Specification
Version	1.0
Date	27 August 2026
Source	Direct repository audit
Scope	Overview-level (see §4.2.2)

Status legend used throughout this document

- **Real** built, wired end-to-end, and tested
- **Partial** real on one side (usually backend) without its full counterpart
- **Deferred** schema or scaffold exists, deliberately not wired up yet
- **Not built** no implementation exists today

Contents

Chapter 1 — Requirements.....	4
1.1 Introduction.....	4
1.1.1 Project overview.....	4
1.1.2 Problem statement.....	4
1.1.3 Purpose of the system.....	4
1.1.4 Intended audience.....	5
1.1.5 Intended use.....	5
1.1.6 Scope of the system	5
In scope — real today.....	5
In scope, but not yet live — explicitly deferred	6
Out of scope	6
1.1.7 Definitions, acronyms, and abbreviations.....	6
1.2 Overall description	7
1.2.1 Software development methodology	7
1.2.2 Assumptions and dependencies.....	7
Assumptions	7
Dependencies	7
Constraints.....	8
Operating environment.....	8
1.3 System features and requirements	8
1.3.1 Functional requirements	8
a) Member functional requirements.....	8
b) Admin functional requirements	8
c) Bank functional requirements	9
d) Telecom functional requirements.....	9
e) Insurance functional requirements.....	9
f) Super-admin functional requirements.....	9
g) Hospital — current status.....	9
Core system processing requirements	9
System interaction requirements.....	10
1.3.2 Non-functional requirements.....	10
Performance.....	10
Usability.....	10
Security.....	10
Reliability	11

Scalability.....	11
Maintainability	11
Compatibility	11
1.3.3 System features.....	11
1.3.4 Data requirements and data sources	12
Data requirements	12
Data sources	12
Chapter 2 — SSAD — Structured System Analysis and Design	13
2.1 Context diagram	13
2.2 Level 0 data flow diagram	13
2.3 Entity relationship diagram (overview)	14
2.4 Logic diagrams	15
2.4.1 Process flowchart — contribution to insurance allocation	15
2.4.2 State diagram — payment transaction lifecycle	15
Chapter 4 — System Design	17
4.1 System architecture.....	17
4.2 Detailed system design.....	17
4.2.1 Physical design.....	17
4.2.2 Database design (overview)	18
4.2.3 Interface design	18

SCOPE NOTE

This SRS follows the structure of the project's approved SRS/SAD template, with **Chapter Three (OOSAD — use-case, class, sequence and deployment diagrams)** and the **Planning / cost-benefit chapter** intentionally excluded. Under §1.3.4, *Data Acquisition* is omitted; *Data Requirements* and *Data Sources* are retained. Because the live schema now spans 23 migrations and several dozen tables, §4.2.2 gives a database *overview* rather than a full column-level data dictionary — the migration files and `database/schema/tujitunze.sql` remain the authoritative reference for that level of detail.

CORRECTION AGAINST PRIOR PROJECT NOTES

This document is grounded in a direct audit of the current source tree, not in previously circulated notes. One correction worth flagging up front: earlier project documentation described a live **Hospital** staff dashboard. No `hospital` backend module, controller, or frontend route group exists in the repository today — `Hospital` survives only as a seeded role name and as read-only reference data (partner hospital directory, claims, verifications) consumed by Member-facing screens. This SRS treats that as the current, honest state of the system; see §1.1.6 and §4.2.3.

Chapter 1 — Requirements

1.1 Introduction

1.1.1 Project overview

Tujitunze (HSIMS — Health Savings and Insurance Management System) is a community-based health savings platform for Tanzania. It lets a member accumulate a personal health fund from the mobile money, airtime, and bank activity they already do every day, and routes a share of each contribution into that member's insurance cover automatically.

The platform is a Next.js (App Router) frontend over a NestJS + TypeORM + PostgreSQL backend, organised around seven operating roles — Member, Admin, Bank, Telecom, Insurance, Super-admin, and the currently-inactive Hospital role — each with its own guarded API surface and, for six of the seven, its own operational dashboard.

1.1.2 Problem statement

- **Low formal health-cover penetration.** Many Tanzanians who transact daily through mobile money and airtime purchases have no mechanism that converts that everyday activity into health savings or insurance cover.
- **Fragmented, untraceable contributions.** Where informal or scheme-based health contributions exist, members typically cannot see, per contribution, where their money went — which channel it came through, which ledger it landed in, or which insurer it was allocated to.
- **Manual reconciliation across partners.** Telecom operators, banks, and insurers each keep their own records; matching a member's contribution across all three has historically been a manual, error-prone process for the operator running the scheme.

1.1.3 Purpose of the system

Tujitunze exists to:

- Turn routine telecom and bank activity into an automatic, rule-computed contribution toward a member's own health wallet.
- Make every contribution traceable end-to-end — from the originating webhook, through the contribution ledger, into the wallet, and (where the member holds an active policy) into a specific insurer's allocation.

- Give each partner in the scheme — telecom operator, bank, insurer, and the platform operator itself — its own tenant-scoped view of exactly its own data, with nothing bled across partners.
- Keep a full audit trail of the sensitive actions taken against member identity, financial, and health data.

1.1.4 Intended audience

Role	Who	What they need from the system
Member	A registered citizen contributing to and drawing on their own health fund	See their wallet, contributions, insurance cover, claims, and notifications
Admin	Internal Tujitunze operations staff	Oversee members, reconcile contributions across channels, produce financial reports, review the audit log
Bank	Staff at a partner bank	Manage their bank's linked accounts, contribution/transaction records, settlements to Tujitunze, and reconciliation
Telecom	Staff at a partner telecom operator	Manage contribution/levy data, member roster, API credentials, and reconciliation for their operator
Insurance	Staff at a partner insurance provider	See claims and contribution allocations routed to their plans, and settlement records
Super-admin	Platform owner/operator	Provision staff accounts for any role, and manage the roles/permissions catalog itself
Hospital not built	Staff at a partner hospital	Seeded as a role and represented as reference data only — see §1.1.6

1.1.5 Intended use

Tujitunze is intended for continuous, browser-based use: a Member interacts with it to build and monitor a personal health fund; Bank, Telecom, and Insurance staff use it as their partner-facing operations console for that scheme; Admin and Super-admin use it to run and govern the platform. Every role signs in once and is routed to a role-scoped dashboard; a staff account with no tenant assigned is refused access to its own dashboard (403) rather than shown someone else's data or an empty screen.

1.1.6 Scope of the system

In scope — real today

- Member registration, authentication, profile, phone/bank-account linking, and health wallet with a real transaction ledger.
- Authenticated, signed webhook intake from Telecom and Bank partners that computes a contribution via a configurable rule engine and credits the wallet — proven live, not simulated.
- Automatic insurance allocation with a fully traceable contribution → wallet transaction → allocation → provider chain.
- Six of seven role-scoped operational dashboards (Member, Admin, Bank, Telecom, Insurance, Super-admin), each backend-guarded and tenant-isolated.
- Cross-channel reconciliation, financial reporting, staff provisioning, and a roles/permissions catalog administered by Super-admin.

- Password reset over email or Tanzanian phone number, rate-limited, OTP-based.

In scope, but not yet live — explicitly deferred

- **Deferred** A Vodacom M-Pesa mobile-money adapter (session handling, C2B/B2C/B2B, idempotent payment-transaction recording) exists up to the point of the real provider call, which is blocked on Vodacom's own endpoint documentation rather than left unbuilt.
- **Deferred** A usage-quantity-based 6% contribution model (voice minutes / SMS / data) — schema only, zero rows, not wired into any application code.
- **Partial** Frontend screens for the Insurance role's claims, settlements, allocation, and reporting endpoints, and for Admin's contribution-summary reporting — the backend is real; no UI consumes it yet.

Out of scope

- **A Hospital staff portal. Not built** No backend module or frontend route group exists for hospital staff. The `hospitals` directory, `healthcare_claims`, and `healthcare_verifications` tables remain as reference data, browsed read-only by Members; clinical workflows (check-in, claim filing, appointments) have no owner in the current build.
- A real external payment rail beyond Tujitunze's own webhook edge — nothing today moves real money at a telecom operator or bank; contribution figures are internal, authenticated bookkeeping until a partner's live systems are pointed at the existing webhook endpoints.
- Production-grade security hardening (security headers, httpOnly session cookies, distributed rate-limit storage) — see §1.3.2.
- Real email/SMS delivery — the code paths exist and fail closed, but no production mail or SMS credentials are configured in this environment.

1.1.7 Definitions, acronyms, and abbreviations

HSIMS	Health Savings and Insurance Management System — the platform this document specifies
NIDA	Tanzania's National Identification Authority; the national ID number captured for each member
JWT	JSON Web Token — the bearer credential issued at login
RBAC	Role-Based Access Control — enforced by <code>JwtAuthGuard</code> + <code>RolesGuard</code> on every guarded route
DTO	Data Transfer Object — a <code>class-validator</code> -annotated request-body shape validated by a global <code>ValidationPipe</code>
ORM	Object-Relational Mapper — TypeORM, used for repository/query-builder access and versioned migrations
C2B / B2C / B2B	Customer-to-Business / Business-to-Customer / Business-to-Business — the three Vodacom M-Pesa payment operation shapes the adapter is built against
MSISDN	Mobile Station International Subscriber Directory Number — a mobile phone number in its international-dialling form
OTP	One-Time Password — used for phone-based password reset
TZS	Tanzanian Shilling
HMAC	Hash-based Message Authentication Code — used to verify inbound webhook authenticity alongside a

	per-partner API key
SSAD / DFD / ERD	Structured System Analysis and Design / Data Flow Diagram / Entity Relationship Diagram — Chapter 2's modelling techniques
Tenant scoping	Restricting a staff account to only the bank/operator/insurer it is linked to, via a nullable foreign key on <code>users</code>

1.2 Overall description

1.2.1 Software development methodology

Tujitunze is built iteratively: the migration history (23 sequential, additive files in `database/migrations/`) shows each unit of work landing as an audited, self-contained increment — a migration's own header typically records the audit finding it addresses, what it deliberately does *not* yet touch, and why. Rather than a single big-bang release, features arrive in phases (for example the payment-transaction layer of migration 0018 was corrected in place by 0019–0023 as the underlying business rule for usage-based contributions was clarified), with each phase re-verified against the running system before the next begins.

Every change is additionally required to pass through a documented six-stage Secure Software Development Life Cycle before it is considered done: **Requirements & Planning** (state the security requirement alongside the functional one, classify the data touched), **Design** (threat-model, least-privilege check), **Implementation** (DTO validation, parameterised queries, guarded endpoints), **Verification** (a security/code review pass, a boundary-proving test — not just a happy path), **Deployment** (CORS allowlist, HTTPS, security headers), and **Maintenance** (audit-log coverage, a living list of known gaps). That known-gaps list is itself treated as a project artefact, not a one-off note — several items closed since baseline are recorded as closed, and what remains open is listed plainly in §1.3.2 rather than omitted.

1.2.2 Assumptions and dependencies

Assumptions

- A member has access to a smartphone or basic phone, an active mobile money/airtime relationship with a partner telecom operator, and/or an account at a partner bank.
- A member holds or can obtain a NIDA number for identity verification at registration.
- Staff accounts for Bank, Telecom, Insurance, Admin, and Super-admin roles are provisioned by a Super-admin, not self-registered.
- Partner telecom operators and banks are willing and able to call Tujitunze's authenticated webhook endpoints from their own systems once integrated.
- Users interact over a standard modern web browser; no native mobile app exists in the current build.

Dependencies

Category	Depends on
Runtime	Node.js server for NestJS (backend) and Next.js (frontend); PostgreSQL for persistence
Identity data	NIDA number format/uniqueness at registration
Contribution channels	Telecom operator and bank webhook delivery, authenticated by a per-tenant API key and HMAC signature
Mobile money	Vodacom's M-Pesa Open API — session/C2B/B2C/B2B endpoint documentation, currently the blocking dependency for live mobile-money contributions
Notifications	An SMTP provider (email reset) and Africa's Talking

	(SMS reset) — both fail closed with 503 when unconfigured, rather than pretending to send
Libraries	TypeORM, <code>class-validator</code> , <code>bcrypt</code> , <code>@nestjs/throttler</code> , <code>@nestjs/jwt</code>

Constraints

- No live mobile-money or bank settlement exists yet; every contribution figure today is internal bookkeeping authenticated at Tujitunze's own edge, not a verified external funds movement.
- Development environments run with `TypeORM synchronize: true` and no production security headers — acceptable for local development, explicitly disqualifying for any non-local deployment until closed (§1.3.2).
- The Hospital role has no functioning staff module — any requirement that assumes hospital-side interaction (claim filing, check-in) is out of scope until that module is built.

Operating environment

- Server: Linux or Windows host running Node.js, fronting a PostgreSQL database.
- Client: any modern desktop or mobile browser (Chrome, Firefox, Edge, Safari) over HTTPS in production.
- Network: internet-connected for both member/staff use and partner webhook delivery.

1.3 System features and requirements

1.3.1 Functional requirements

Requirements are grouped by role. Each line carries a status chip reflecting what exists in the codebase today, not an aspiration.

a) Member functional requirements

- **Real** — The system shall let a citizen register with NIDA number, phone number, and password, and authenticate via JWT.
- **Real** — The system shall let a member view and update their profile, and link/unlink phone numbers and bank accounts.
- **Real** — The system shall show a member their health wallet balance, top up the wallet directly, and list every wallet transaction with its contribution channel (airtime, bank transfer).
- **Real** — The system shall show a member their own insurance plan and claim history, and a membership summary combining identity, wallet, and cover.
- **Real** — The system shall notify a member of a wallet top-up, a new linked phone/bank account, a password change, and other sensitive account events.
- **Real** — The system shall let a member reset a forgotten password via email token or phone OTP.
- **Partial** — The system shall provide a mobile-money account onboarding form; the underlying live mobile-money debit is not yet connected (§1.1.6).
- **Not built** — Wallet transfer, wallet withdrawal, telecom airtime/bundle purchase-through-wallet, a member-facing QR code, and a member reports screen are placeholders with no backend today.

b) Admin functional requirements

- **Real** — The system shall let Admin manage members and hospital-directory reference records, and view/adjust member status.
- **Real** — The system shall give Admin a contributions summary and filterable list across channels and statuses, and cross-channel reconciliation checks/runs.
- **Real** — The system shall give Admin financial reports with filters and a transaction drill-down.
- **Real** — The system shall let Admin read the platform audit log.

- **Partial** — Contribution filtering is channel- and status-based only today; filtering by member, date range, or specific operator/bank is not yet implemented.

c) Bank functional requirements

- **Real** — The system shall let Bank staff manage their own bank's profile, contact details, and API/webhook credentials, scoped to their `bank_id` tenant link.
- **Real** — The system shall accept an authenticated, signed webhook from the bank recording a contribution, and let staff manually record, reverse, or fail a transaction.
- **Real** — The system shall let Bank manage Tujitunze's own fund accounts and fund transfers held at that bank, and create/complete settlements to Telecom or Hospital partners.
- **Real** — The system shall give Bank a three-way reconciliation (Matched / Discrepancy / Unmatched), reports, and activity/API access logs — every page here has a matching frontend screen.

d) Telecom functional requirements

- **Real** — The system shall mirror Bank's operator-profile, credentials, contribution recording/reversal, reconciliation, reports, and audit-log capabilities for a Telecom operator.
- **Real** — The system shall let Telecom staff view their contribution rules (rate, minimum amount, effective date) read-only.
- **Partial** — The system shall expose a Vodacom M-Pesa transaction status query and reversal endpoint; the underlying live provider call is blocked on Vodacom's endpoint documentation (§1.1.6).

e) Insurance functional requirements

- **Real** — The system shall let Insurance staff view claims routed to their provider and update claim status.
- **Real** — The system shall let Insurance staff view per-contribution allocation records with a full traceability chain back to the originating contribution, plus settlement and reporting endpoints.
- **Not built** — None of the above beyond the summary dashboard has a frontend screen yet — every Insurance capability past the landing dashboard is API-only today.

f) Super-admin functional requirements

- **Real** — The system shall let Super-admin provision a staff account for any non-Member role, validating NIDA/email uniqueness and the correct tenant-link requirement for that role.
- **Real** — The system shall let Super-admin list every staff account and the available tenants for the provisioning form.
- **Real** — The system shall let Super-admin create, rename, delete, and assign permissions to roles, while rejecting a rename/delete of the seven core seeded role names.
- **Not built** — There is no endpoint to edit, deactivate, or reassign a staff account after creation.

g) Hospital — current status

HONEST GAP, NOT A PARTIAL FEATURE

`Hospital` is seeded as a role and `users.hospital_id` still exists as a tenant-link column, but no backend module, controller, or frontend route group implements it. A hospital cannot log in and reach a working dashboard today. The `hospitals` table, `healthcare_claims`, and `healthcare_verifications` persist only as reference data, read (never written) by Member-facing endpoints for hospital-directory browsing and verification history.

Core system processing requirements

- **Real** — The system shall authenticate inbound partner webhooks by a per-tenant API key plus an HMAC signature, independent of member JWT auth, via dedicated webhook-intake controllers.
- **Real** — The system shall compute each contribution amount from a configurable rule (percentage rate, minimum amount, effective date) rather than a hardcoded constant.

- **Real** — The system shall guarantee idempotency on a redelivered webhook — the same external transaction reference shall never produce a second ledger row or a second wallet credit.
- **Real** — The system shall credit the wallet and, when the member holds an active policy, create an insurance allocation in the same transaction as the contribution — never as two separate, possibly-inconsistent writes.
- **Real** — The system shall record a sensitive write (identity change, financial write, status change, staff/role management action) to the audit log inside the same database transaction as the write itself.

System interaction requirements

- **Real** — The system shall serve every role through a single web platform, routing each role to its own path-prefixed dashboard (for example `/bank/dashboard`, `/telecom/dashboard`) to avoid Next.js route-group collisions.
- **Real** — The system shall enforce every role boundary on the backend (`JwtAuthGuard` + `RolesGuard`); the frontend's route-group gate is UX/defense-in-depth only, never the actual authorization boundary.
- **Real** — The system shall scope a tenant role's data (Bank/Telecom/Insurance) to its own linked institution and refuse (403) rather than return empty or global data when that link is unset.

1.3.2 Non-functional requirements

Performance

- Dashboard and list endpoints shall respond within normal interactive latency (low hundreds of milliseconds) under expected single-instance load; no dedicated load-testing has been run against this build.
- Webhook intake shall process a single contribution synchronously within one request/response cycle, including idempotency check, rule computation, ledger write, and allocation.

Usability

- Each role shall share one consistent dashboard shell (sidebar + layout) so navigation is uniform across every page within that role's route group.
- A role's navigation shall only list pages that actually exist, to avoid dead links into unbuilt route-group stubs.

Security

Implemented today: JWT + role-based guards on every role-scoped route; tenant scoping by nullable foreign key; DTO validation via a global `ValidationPipe`; parameterised/query-builder database access throughout; rate limiting on login, registration, phone-number, and super-admin write endpoints; an explicit CORS allowlist; password reset tokens/OTPs stored only as salted hashes with attempt limits and expiry; audit logging on the sensitive write paths listed in §1.3.1.

Still open — treated as required work before any non-local deployment, not silently dropped:

- **Open** — No `helmet`/security-header middleware is configured.
- **Open** — The JWT is stored in browser `localStorage`, not an `httpOnly` cookie — exposed to XSS until session infrastructure is built.
- **Open** — `database.config.ts` falls back to a default DB password if `DB_PASSWORD` is unset, instead of failing closed.
- **Open** — `TypeORM synchronize: true` remains set — acceptable only for local development.
- **Open** — Rate-limit state is stored in-process and will not be shared once the backend runs behind a load balancer with multiple instances.
- **Open** — The audit log has no pagination or filtering beyond the latest 200 rows.

- **Open** — Telecom/Bank `webhook_secret` is stored retrievable rather than hashed, because Tujitunze needs the raw value to sign outgoing deliveries — a real plaintext-at-rest gap, currently unused by any live code path.

Reliability

- A failed or ambiguous provider response (network error, timeout, unknown status) shall never be recorded as a successful payment; only a confirmed provider response may mark a transaction **SUCCESSFUL** — enforced at the database level, not only in application code.
- Email and SMS delivery shall fail closed (**503**) when unconfigured rather than silently pretending to have sent something.

Scalability

- The contribution rule engine, contribution channels, and provider adapters are designed as configuration/data rather than hardcoded constants, so a new operator, bank, or rule can be added without a code change to the core ledger path.
- Horizontal scaling of the backend is currently constrained by the in-process rate-limiter and session-key cache (§ above) until a shared store is introduced.

Maintainability

- Schema changes are additive, versioned SQL migrations under `database/migrations/`, each documenting the audit finding it addresses and what it deliberately leaves untouched.
- Backend test coverage includes 16 end-to-end spec files and 14 unit spec files exercising authorization boundaries, webhook idempotency, reconciliation, and rate limiting — not just happy paths.

Compatibility

- The frontend targets evergreen desktop and mobile browsers via Next.js's standard support matrix; no legacy-browser support is targeted.

1.3.3 System features

At its centre, Tujitunze gives every member a **unified health wallet**: one balance, backed by an immutable transaction ledger, regardless of which channel funded it. Feeding that wallet is a **multi-channel contribution engine** — authenticated webhooks from telecom and bank partners, each run through a configurable rate/minimum rule before a single shared crediting path touches the ledger, so airtime-based and bank-transfer-based contributions can never diverge in how they're recorded. A mobile-money channel via Vodacom M-Pesa is built up to the same idempotent, database-enforced-honesty standard, pending the provider's own endpoint documentation.

Where a member holds an active policy, a contribution automatically produces an **insurance allocation**, and that allocation is traceable in one query back to the exact contribution and wallet entry that produced it — a design goal the platform can answer concretely today ("where did this member's TZS 20 go?"), not just assert. Sitting over all of this is **cross-channel reconciliation**: Bank and Telecom can each upload or generate a batch of their own records and have it matched against Tujitunze's ledger, surfacing matches, discrepancies, and unmatched rows.

Six **role-scoped operational dashboards** give each class of user — Member, Admin, Bank, Telecom, Insurance, Super-admin — only the views and actions that belong to it, backed by the same guard pattern everywhere. A platform-wide **audit trail** and a **notifications** system keep members and operators aware of sensitive events as they happen. Finally, **Super-admin governance** closes the loop: staff accounts and the roles/permissions catalog itself are managed through the same audited, guarded pattern as every other sensitive write, rather than hand-written SQL.

1.3.4 Data requirements and data sources

Data requirements

DATA CLASSIFICATION

Tujitunze handles four categories of sensitive data, each requiring an authorization check and an audit trail: **PII** (name, NIDA number, address, date of birth), **financial** (wallet, bank, telecom, and mobile-money transactions), **health** (insurance policies, claims, verifications), and general **administrative** data (roles, permissions, tenant links).

Beyond identity and financial data, the system requires: contribution rule configuration (rate, minimum, effective date per rule type); reference data for partner institutions (telecom operators, banks, insurance providers and their plans); and operational data generated by the platform itself — audit log entries, notifications, and reconciliation run records.

Data sources

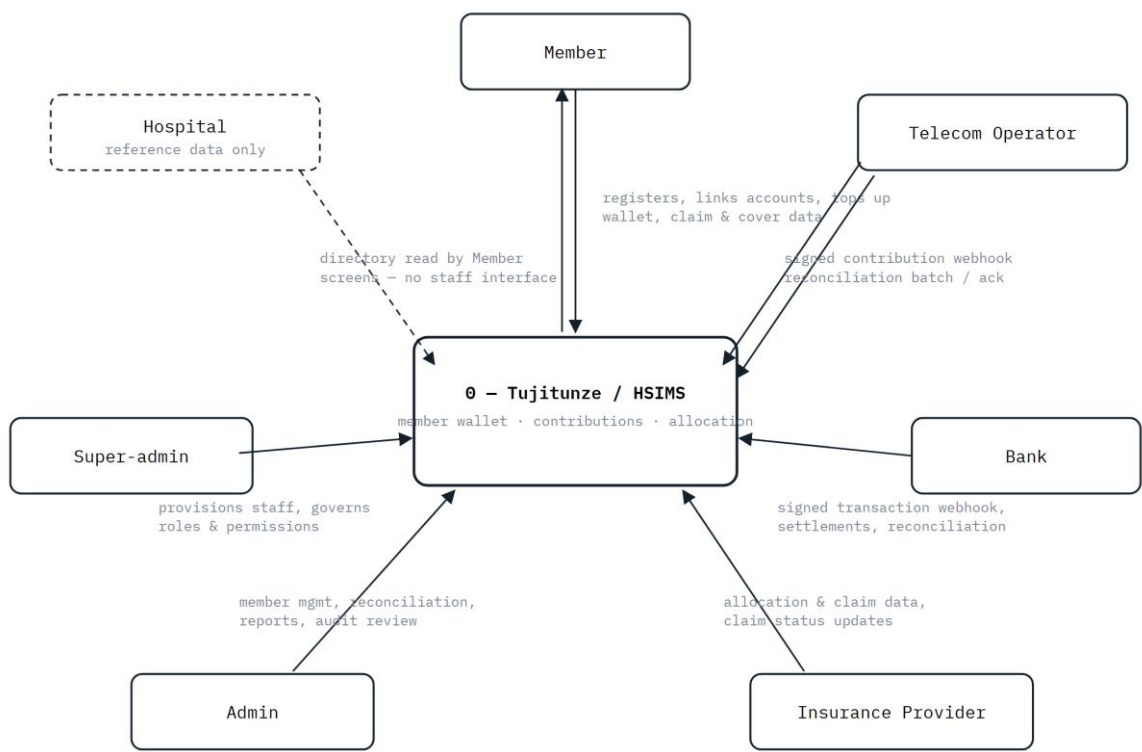
- **Member self-registration** — identity, contact, and initial phone/bank-account linkage, entered directly by the member.
- **Super-admin provisioning** — every non-Member staff account and its tenant link.
- **Admin-managed reference data** — the hospital directory and member-record corrections.
- **Authenticated partner webhooks** — real, live contribution notifications from Telecom and Bank systems, the primary source of ongoing financial transaction data.
- **Vodacom M-Pesa gateway** — the mobile-money channel's intended data source once the adapter's remaining wire call is unblocked.
- **Seed/reference data** — insurance providers and starter plans (NHIF, Diamond Trust Insurance, Jubilee Insurance) and contribution-rate rules, loaded via versioned migrations so they are reproducible across environments.

Chapter 2 — SSAD — Structured System Analysis and Design

2.1 Context diagram

The context diagram treats Tujitunze as a single process and shows every external party that exchanges data with it. Six parties actively interact with the system; a seventh, Hospital, is drawn with a dashed outline because it is present only as reference data consulted by the system, not as an active participant (§1.1.6).

FIGURE 1

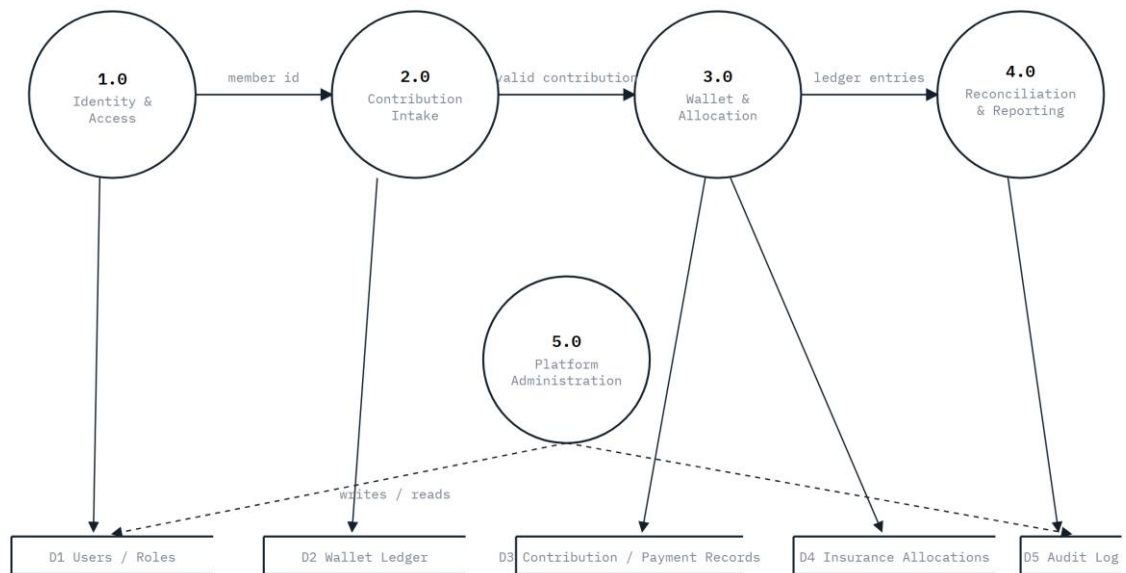


The system boundary and its six active external entities, plus Hospital as a dashed, read-only reference-data source rather than an active participant.

2.2 Level 0 data flow diagram

Decomposing the single context-diagram process into its five major internal processes and the data stores they read and write.

FIGURE 2

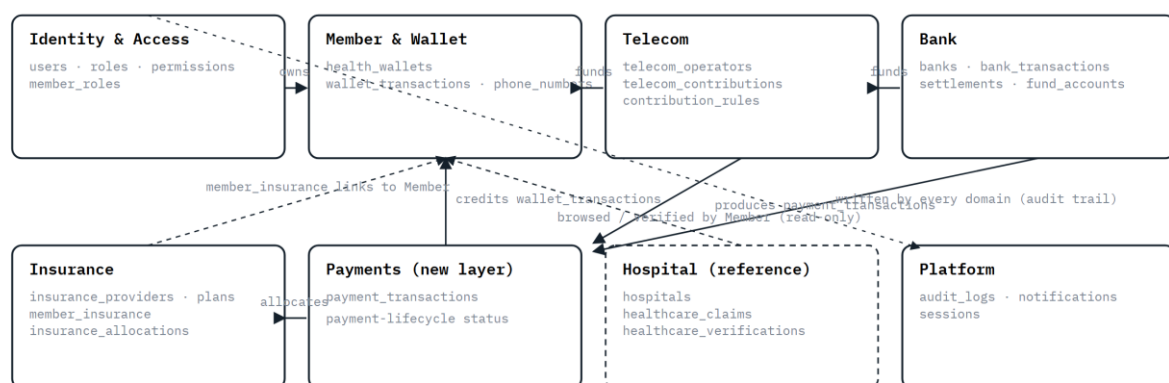


1.0 authenticates and links a role/tenant to a request; 2.0 verifies and rule-computes a webhook contribution; 3.0 credits the wallet and creates an insurance allocation in one transaction; 4.0 matches partner batches against the ledger and produces reports; 5.0 (Platform Administration — staff and role governance) writes to every store, shown dashed since it spans identity, ledger, and audit data rather than owning one store exclusively.

2.3 Entity relationship diagram (overview)

Given the size of the live schema (23 migrations, several dozen tables), this ERD groups tables by domain rather than listing every column — see §4.2.2 for how this maps onto physical database design.

FIGURE 3

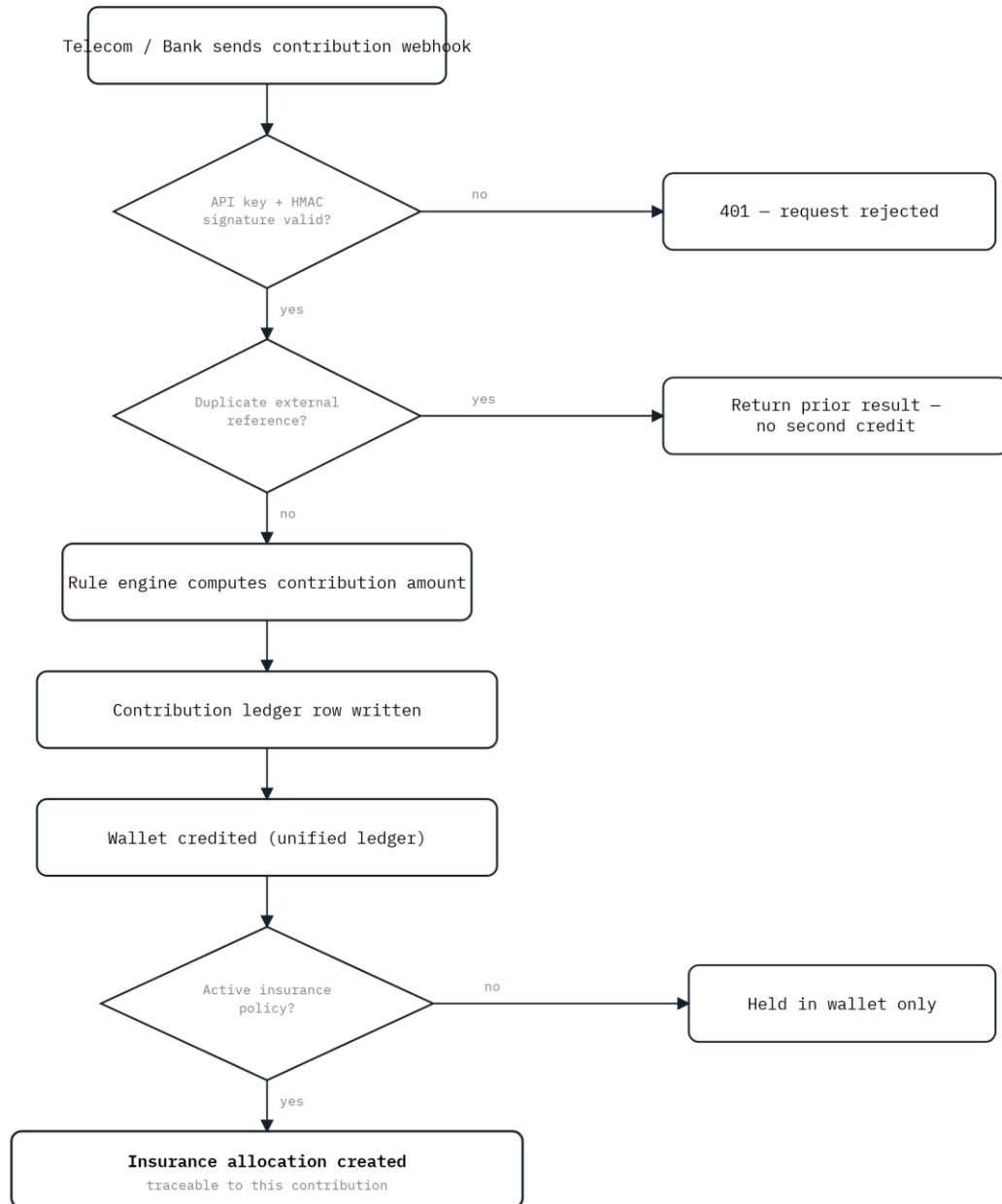


Domain-grouped entity overview. Solid arrows are the live ledger chain (Telecom/Bank → Payments → Member & Wallet → Insurance); dashed arrows are read-only or cross-cutting relationships. Full column-level detail lives in the migration files, not in this document.

2.4 Logic diagrams

2.4.1 Process flowchart — contribution to insurance allocation

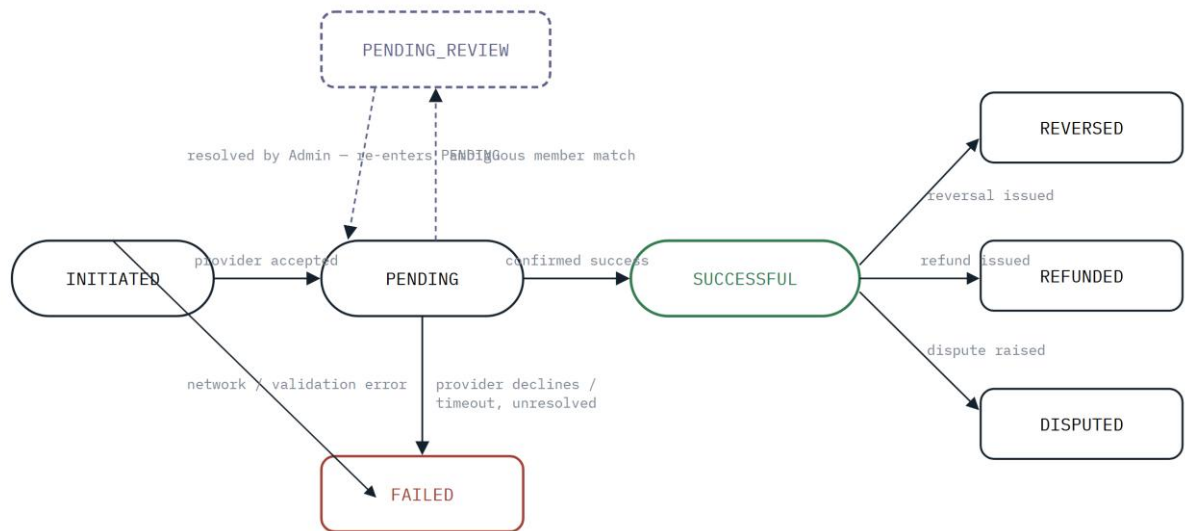
FIGURE 4



Every inbound contribution passes signature verification and an idempotency check before the rule engine, ledger write, wallet credit, and — where a policy is active — insurance allocation all occur in one transaction.

2.4.2 State diagram — payment transaction lifecycle

FIGURE 5



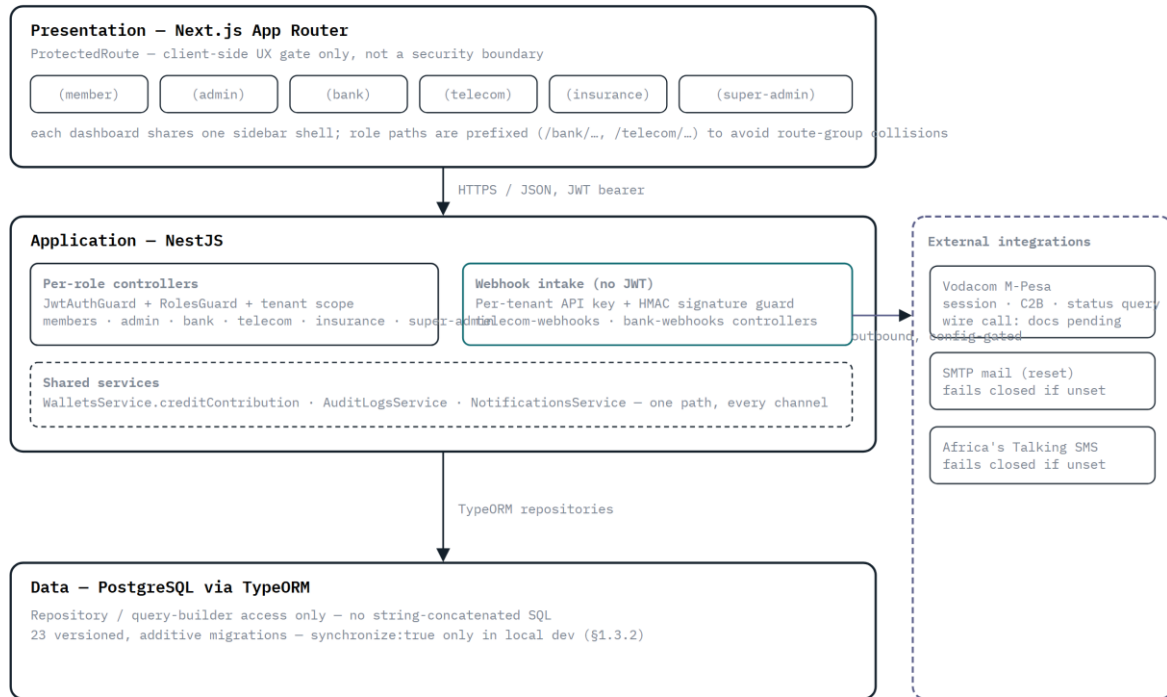
*The eight-value payment status vocabulary `payment_transactions` is built around. A database constraint enforces that only a real provider transaction ID can move a row to **SUCCESSFUL** — the state machine cannot be talked into a false-positive success by application code alone.*

Chapter 4 — System Design

4.1 System architecture

Tujitunze follows a layered architecture: a Next.js presentation tier, a NestJS application tier enforcing every real authorization decision, a PostgreSQL data tier reached only through TypeORM, and an external-integration edge that fails closed whenever a provider isn't configured.

FIGURE 6



Every real authorization decision lives in the application tier, not the frontend gate. Webhook intake is a deliberately separate, non-JWT path with its own credential model. External providers are called from the application tier and fail closed (503) rather than silently no-op when unconfigured.

4.2 Detailed system design

4.2.1 Physical design

Layer	Requirement
Application server	Node.js runtime hosting the NestJS backend and the Next.js frontend (or a Next.js edge/server deployment target)
Database server	PostgreSQL instance, reachable from the application server, holding all persistent state
Client	Any modern desktop or mobile browser; the frontend is responsive, no native app
Network	HTTPS in any non-local deployment; explicit CORS allowlist configured in <code>main.ts</code> ; inbound webhook traffic from partner telecom/bank systems
Development tooling	NestJS CLI, TypeORM migration runner, Jest (unit + e2e), npm — the diagrams in this document are authored directly as part of the specification rather

4.2.2 Database design (overview)

WHY THIS SECTION STOPS AT AN OVERVIEW

The live schema has grown past the point where a column-by-column data dictionary belongs in a requirements document: 23 sequential migrations on top of the base schema, spanning identity, wallet, telecom, bank, insurance, payments, and audit domains. Per the scope agreed for this document, this section describes the design at the level of table groups and the relationships between them (mirroring §2.3's ERD); `database/schema/tujitunze.sql` and `database/migrations/` remain the single source of truth for exact columns, types, and constraints.

The most consequential design decision in the schema is the four-layer contribution chain, added incrementally rather than designed monolithically up front:

1. **telecom_contributions / bank_transactions** — the original per-channel contribution ledger, workflow-staged (Pending → Received → Validated → Allocated → Failed → Reversed).
2. **wallet_transactions** — the unifying ledger every channel credits through one shared method; the wallet's displayed balance is always a derived total, never itself the source of truth.
3. **insurance_allocations** — created in the same transaction as a contribution credit whenever the member holds an active policy, giving a provably traceable contribution → wallet entry → allocation → provider chain.
4. **payment_transactions** — a newer layer sitting above the other three, recording a provider's own payment-lifecycle status (`INITIATED/PENDING/SUCCESSFUL/FAILED/REVERSED/REFUNDED/DISPUTED/PENDING_REVIEW`) distinctly from the ledger's own workflow vocabulary — see Figure 5.

A related, deliberately unwired addition is `telecom_usage_events`: a table for a usage-quantity-based contribution model (6% of voice minutes / SMS / data consumed, converted to TZS only at a provider-authorized rate — never an invented per-unit price). It holds zero rows and is not read by any controller today; it exists as a reviewed, corrected schema ahead of the application code that will use it.

All schema changes are additive SQL migrations, never a destructive rewrite of a table already carrying data — several migrations in the history explicitly verify "zero rows, safe to correct in place" before altering a table, and none drops or recreates a table that held live data.

4.2.3 Interface design

Every staff role shares one dashboard shell — a persistent sidebar plus a consistent page layout — so a role's navigation stays uniform across every page in its route group, not only its landing dashboard. A route group only lists a navigation entry for a page that actually exists, to avoid linking into an unbuilt stub.

Member Real, most complete

- Dashboard, membership, profile, notifications
- Wallet, wallet/deposit, wallet/transactions
- Insurance/claims, insurance/plans, verifications
- Onboarding/mobile-money

Admin Real

- Dashboard, reports, audit-logs
- Members (list + detail)

Bank Real, matches backend

- Dashboard, profile, fund-accounts
- Transactions, settlements
- Reconciliation (+ detail), reports, audit-logs

Telecom Real, matches backend

- Dashboard, operator, members
- Contributions, contribution-rules
- Reconciliation (+ detail), reports, audit-logs

Insurance Dashboard only

- Dashboard — real
- Claims, settlements, allocations, reports: backend real, **no frontend page yet**

Super-admin Real

- Dashboard
- Administrators (provisioning)
- Roles & permissions

Still placeholder on the Member side: QR, reports, telecom airtime/bundles/payments, wallet transfer, and wallet withdraw render a coming-soon state with no backend behind them.

HOSPITAL INTERFACE — NONE

There is no ([hospital](#)) route group and no page a hospital staff member could sign into. Hospital-related interface today is entirely Member-facing: a read-only hospital directory and a member's own verification history, both sourced from the reference tables in §2.3.

Prepared from a direct audit of the Tujitunze/HSIMS repository (backend, frontend, and database/migrations/) on 27 August 2026. Where this document and previously circulated project notes disagreed, the live source tree was treated as authoritative and the discrepancy was called out explicitly rather than silently reconciled.